ROOTCAUSE

F-DBTOOL-2 — Root Cause + Fix: `update --description` desyncs table-representation items

Revision: 1 **Last modified:** 2026-07-12T00:00:00Z **Status:** Fixed (validated on DB copies only — no live-DB write, no commit performed by this investigation)

Scope note (§11.4.6 / zero-risk constraint)

This investigation operated **exclusively on copies** of `docs/workable_items.db`. The live database (`/home/milos/Factory/projects/tools_and_research/helix_code/docs/workable_items.db`) was **never opened for writing**. Its md5sum before and after this session is identical: `187b0a51a86be4b2fa22dfca2311d61a`. **No `git commit` was executed.** The fix is a working-tree edit to `constitution/scripts/workable-items/cmd/workable-items/mutate.go` plus a new regression-test file, both uncommitted at the time of writing.

Context

F-DBTOOL-1 (already committed at constitution submodule HEAD `3302587`) fixed a representation-scoping bug in `loadItem`/`update`/`close`/`reopen`/`block`/`obsolete-details`'s `WHERE` clauses. F-DBTOOL-2 is a **separate, still-open** bug in the same tool: applying the 31 `field=="description"` hygiene proposals from `docs/qa/fdbtool_hygiene_20260712T071056Z/W2C_hygiene_proposals.jsonl` (identical content to `docs/qa/discovery_hardening_wave2_20260711T214502Z/W2C_hygiene_proposals.jsonl`) via `wi update --id <atm> --description <val> --location Fixed`, then `sync db-to-md` + `diff`, produced ~81 spurious differences (38 "body differs", 31 "present in DB, absent in Markdown", 12 status/type mismatches), all on `FIX-2026-05-19#N` items.

Phase 1 — Reproduction (minimal)

Built the tool (`go build -o /tmp/wi2 ./cmd/workable-items`), copied the live DB to a scratch path, and applied **a single** description update:

```
$ /tmp/wi2 update --id "FIX-2026-05-19#10" --db f2_copy.db --location Fixe
    --description "This task added translation support to five short help
update: FIX-2026-05-19#10 updated in Fixed (status=Implemented (→ Fixed.md

$ /tmp/wi2 sync db-to-md --db f2_copy.db --out-issues out_Issues.md --out-
$ /tmp/wi2 diff --db f2_copy.db --issues out_Issues.md --fixed out_Fixed.m
~ FIX-2026-05-19#10 body differs (md=214 bytes db=387 bytes)
~ FIX-2026-05-19#11 body differs (md=261 bytes db=214 bytes)
~ FIX-2026-05-19#12 body differs (md=187 bytes db=261 bytes)
~ FIX-2026-05-19#13 body differs (md=217 bytes db=187 bytes)
... (64 lines total, cascading through every later same-dated row)
```

A **single** targeted update already produces the full cascading-diff signature — confirming the bug is triggered by the shape of the mutation itself, not by volume.

The DB query for the mutated item **before** the update:

```
atm_id             current_location  representation  status
FIX-2026-05-19#10  Fixed             table            Implemented (→ Fixed
body_md = "| 2026-05-19 | panoptic × 5 cobra Short descriptions migration
```

`FIX-2026-05-19#10` is a **`representation='table'`** item — a legacy Fixed.md pipe-table closure row (`| date | title | type | status | round | commit(s) | evidence |`), parsed by `parseFixed`'s `emitLegacyTable` (`cmd/workable-items/parse.go:802-865`). It has no H2 heading and no `**Status:**` block; body_md is the single verbatim pipe-row line.

Phase 2 — Root cause (FACT, cited file:line)

Defect A — `update --description` ignores `representation` when regenerating `body_md`

`cmd/workable-items/mutate.go`, `updateCmd`, pre-fix lines 170-172:

```
var newBody string
if set["description"] {
    newBody = renderItemBody(cur.AtmID, cur.Title, cur.Type,
        cur.Severity, cur.Description, cur.Status,
        cur.CreatedBy, cur.AssignedTo)
} else {
    ...
}
```

`renderItemBody` (`cmd/workable-items/crud.go:528-544`) **unconditionally** emits the H2-section shape:

```
fmt.Fprintf(&b, "## %s — %s\n\n", id, title)
fmt.Fprintf(&b, "**Status:** %s\n", status)
fmt.Fprintf(&b, "**Type:** %s\n", typ)
...
```

For a `representation='table'` item this REPLACES the single pipe-row line with a multi-line H2 block — a shape the pipe-table structure of Fixed.md never expects at that position (`renderDocument`, `cmd/workable-items/db.go:632-696`, looks up `bodyByKey[atmID+"\x00"+rep]` for an `item`-kind `doc_segment` and appends it verbatim via `appendSegment` — it has no knowledge of what shape that body "should" be).

Defect B — the regenerated heading is unparseable, AND same-dated legacy rows are positionally (not stably) identified

Two cooperating consequences fire once the wrong-shaped body is re-parsed by `parseFixed`:

B1 — unparseable heading. `FIX-2026-05-19#10`'s injected line `## FIX-2026-05-19#10 — <title>` matches **none** of parseFixed's heading shapes: - `issueHeadingRe` (`parse.go:21`): `^## ([A-Z]{3}-[0-9A-Za-z]+)(?: \([^)]*\))? — (.+)$` — the capture group `[0-9A-Za-z]+` stops at the SECOND `-` (in `-05-19`), so nothing after `FIX-2026` can complete the match; the trailing `#10` guarantees failure regardless. - `atmShape1HeadingRe`/`atmShape2HeadingRe`/`atmShape3HeadingRe` (`parse.go:59-61`): require a trailing `.`, a bracketed `[XXX-nnn]`, or a leading `§` respectively — none present.

So the injected block is never recognised as a heading; it is absorbed into `rawBuf` as raw prose (`parseFixed`, `parse.go:735-745`), and `emitLegacyTable` (`parse.go:802-865`) finds no `fixedRowRe`-matching lines inside it either — the mutated item **stops existing as an item** on re-parse.

B2 — positional (non-stable) synthetic IDs cascade the damage. `emitLegacyTable`, `parse.go:820-836`:

```
titleCell := m[2]
idm := fixedTitleIDRe.FindStringSubmatch(titleCell)
var id string
if idm != nil {
    id = idm[1]
} else {
    // No ticket id (legacy narrative rows). Synthesize a stable,
    // document-unique key so the row still round-trips +
        validates.
    id = "FIX-" + m[1] // date-based
}
```

```
if n, ok := seenID[id]; ok {
    seenID[id] = n + 1
    id = id + "#" + itoa(n+1)
} else {
    seenID[id] = 0
}
```

None of the `FIX-2026-05-19#N` rows' title cells start with a leading `[A-Z]{3}-[0-9A-Za-z]+` ticket id (`fixedTitleIDRe`, `parse.go:702`), so `idm` is `nil` for every one of them, and `id` is synthesized as `"FIX-" + <date>` — **identical for every row sharing that date** — then disambiguated purely by an **in-scan occurrence counter** (`seenID`). This counter is **not a stored, content-addressed identifier**; it is recomputed from scratch, in top-to-bottom scan order, on every `parseFixed` call. The comment "Synthesize a **stable**, document-unique key" is aspirational, not actual: stability holds only as long as every same-dated row keeps matching `fixedRowRe` in the same order.

Once defect B1 removes one row from that matching sequence (it is no longer line-shaped as a pipe row at all), every **later** same-dated row's occurrence count shifts down by one, so its re-derived id collides with the id the corrupted row used to hold. Concretely, with rows R1..Rn on date `2026-05-19` (R1 → id `FIX-2026-05-19`, Rk → id `FIX-2026-05-19#(k-1)` for k≥2): corrupting R11 (id `#10`) means the re-parse's 11th **recognised** occurrence is now R12 (whose *content* is unchanged) — so it gets re-labeled `#10`, R13 gets re-labeled `#11`, etc. This exactly matches the observed diff:

```
~ FIX-2026-05-19#10 body differs (md=214 bytes db=387 bytes)   <- md[#10]
~ FIX-2026-05-19#11 body differs (md=261 bytes db=214 bytes)   <- db[#11]=
```

`db=214` on the `#11` line is byte-identical to the untouched R12's stored `body_md` length reported on the `#10` line as `md=214` — confirming the shift empirically, not by inference.

Why "31 absent" + "38 body differs" + "12 status/type mismatches" (bulk case)

Each of the 31 targeted updates independently corrupts its own row (defect A), and each corrupted row independently triggers a downstream one-position shift for every later same-dated row (defect B) — the shifts compound across the batch (a later corrupted row's shift interacts with an earlier one's), producing the ~81-line aggregate signature reported in the task.

Phase 3 — Fix (minimal, backward-compatible)

File touched: `constitution/scripts/workable-items/cmd/workable-items/mutate.go` (single-condition change, ~30 lines of explanatory comment added).

```
// F-DBTOOL-2 fix (2026-07-12): ...
var newBody string
if set["description"] && cur.repOrDefault() != "table" {
    newBody = renderItemBody(cur.AtmID, cur.Title, cur.Type,
        cur.Severity, cur.Description, cur.Status,
        cur.CreatedBy, cur.AssignedTo)
} else {
    newBody = cur.BodyMD
    if set["title"] {
        newBody = replaceHeadingTitle(newBody, cur.AtmID,
        cur.Title)
    }
    newBody = canonicalizeBodyStatusLine(newBody, cur.Status)
}
```

Rationale for correctness/safety:

- `items.description` (the DB column) is **still updated** unconditionally earlier in `updateCmd` (`cur.Description = *description`) — the field update itself is not lost, only the `body_md` regeneration is skipped for `representation='table'` items.
- A pipe-table row has **no dedicated description cell** to receive the new text — `Description` is *derived* from `Title` + the `Evidence` cell at parse time (`deriveDescription`, called from `emitLegacyTable`, `parse.go:859`), so there is no lossless place to inject an updated description into the row's single-line shape without inventing new structure. Preserving `body_md` verbatim (the same field-only path already used, and already proven safe, for `--severity`/`--status`-only updates) is the correct minimal behaviour.
- `replaceHeadingTitle` (`crud.go:526-539`) requires a line prefixed `## <id> —`; a pipe-row body has no such line, so it is a no-op (returns body unchanged) — verified by inspection of its guard condition.
- `canonicalizeBodyStatusLine` (`parse.go:496-516`) requires a `**Status:**`-prefixed line (via `lastBodyStatus` → `metaLineRe`, which matches `^\*\*([A-Za-z-]+):\*\*`); a pipe-row body has no `**`-bold text at all, so `lastBodyStatus` returns `("", false)` and the function is a no-op — verified by inspection.
- Does not touch, weaken, or interact with the F-DBTOOL-1 representation-scoped `WHERE` clause (already correct) or the close/obsolete-details paths.

Phase 4 — Validation (copies only)

All commands below ran against **scratch copies** of the live DB under `/tmp/.private/.../scratchpad/f2work/`. The live DB was re-checksummed after this session and is unchanged (`187b0a51a86be4b2fa22dfca2311d61a`).

4a. Tool's own test suite

```
$ cd constitution/scripts/workable-items && go test -count=3 ./...
ok      github.com/HelixDevelopment/HelixConstitution/scripts/workable-ite
```

3 consecutive runs, all green (§11.4.50 deterministic-consistency).

4b. New regression test (§11.4.115 reproduce-first)

New file: `constitution/scripts/workable-items/cmd/workable-items/f_dbtool2_description_table_representation_test.go`

- `TestUpdateDescription_TableRepresentation_RenderItemBodyCorruptsRoundT` — RED / load-bearing companion. Reproduces the exact PRE-FIX behaviour (calls `renderItemBody` directly against a table-representation item, bypassing the new guard) on a 3-row same-dated pipe-table fixture, and proves the third row's content gets relabeled under the second row's id after a `sync db-to-md` + re-parse — i.e. proves this guard is load-bearing, not a tautology.
- `TestUpdateDescription_TableRepresentation_RoundTripsCleanly` — GREEN confirmation. Drives the real `updateCmd` → `syncDBToMD` → `diffCmd` production subcommands end-to-end and asserts `diffCmd` returns `exitOK` (zero differences), and that `items.description` was genuinely updated while body_md stayed the untouched pipe-row line.

```
$ go test -count=1 -run 'TestUpdateDescription_TableRepresentation' -v ./c
=== RUN   TestUpdateDescription_TableRepresentation_RoundTripsCleanly
--- PASS: TestUpdateDescription_TableRepresentation_RoundTripsCleanly (0.0
=== RUN   TestUpdateDescription_TableRepresentation_RenderItemBodyCorrupts
    ...RED reproduced: after a pre-fix renderItemBody UPDATE on a table-re
    db-to-md + re-parse relabels the THIRD row's original content under th
    ("FIX-2026-05-19#1") — proving the table-representation guard in updat
    load-bearing, not a tautology
--- PASS: TestUpdateDescription_TableRepresentation_RenderItemBodyCorrupts
PASS
```

4c. Full 31-description batch on a fresh copy of the live DB

```
$ cp docs/workable_items.db f2_copy_final.db
$ python3 <apply all 31 field=="description" proposals via /tmp/wi2 update
applied 31

$ /tmp/wi2 sync db-to-md --db f2_copy_final.db --out-issues final_Issues.m
wrote .../final_Issues.md (101900 bytes)
wrote .../final_Fixed.md (166020 bytes)

$ /tmp/wi2 diff --db f2_copy_final.db --issues final_Issues.md --fixed fin
diff: DB and Markdown are in sync

$ /tmp/wi2 validate --db f2_copy_final.db
validate: OK — 324 items, all invariants satisfied

$ md5sum docs/workable_items.db
187b0a51a86be4b2fa22dfca2311d61a   <- unchanged from before this session
```

diff = 0 confirmed. validate = OK confirmed. Live DB untouched confirmed.

4d. No regression on the F-DBTOOL-1 fix or severity/close hygiene

The full 31-description batch above ran against a DB that already carries the (previously-applied, per the task framing) severity hygiene + F-DBTOOL-1 representation-scope fix; `diff`/`validate` sweep the **entire** DB, not just the 31 touched rows, and both stayed green — no regression introduced.

Deliverable summary

- **Root cause:** `mutate.go` `updateCmd`'s `--description` path called `renderItemBody` (H2-section shape) unconditionally, corrupting `representation='table'` pipe-row items; `parse.go`'s `emitLegacyTable` derives un-ID'd legacy rows' ids **positionally** (occurrence-counted at parse time, not content-addressed), so one corrupted row cascades a relabeling shift across every later same-dated row.
- **Fix exists and is validated:** yes — `mutate.go`, single-condition change (`cur.repOrDefault() != "table"`), ~30 lines including explanatory comment.
- **Copy-validated proof:** 31 descriptions → `diff: DB and Markdown are in sync` (0 differences) + `validate: OK — 324 items, all invariants satisfied`.
- **Regression tests:** `TestUpdateDescription_TableRepresentation_RoundTripsCleanly`

(GREEN) + `TestUpdateDescription_TableRepresentation_RenderItemBodyCorruptsRoundT` (RED/load-bearing).
- **No live-DB writes. No git commits.**